

Design Overview

In this project I implemented a fault-tolerant linearizable replicated datastore using ZooKeeper. We use a shared log model where all client requests are serialized into sequential persistent znodes. This serialization creates a total global order, ensuring that all replicas process the same operations in the exact same order. Each server replica has a dedicated watcher thread that polls this log and executes requests strictly in a monotonic order, enforced by an atomic 'lastExecutedSeq' counter and a gap detection logic that stops processing if a sequence number is skipped. This prevents replicas from drifting apart even under ZooKeeper notification delays, reordering, or network asynchrony.

State persistence is handled by Cassandra, and every executed operation is applied atomically to the local database before the replica advances its checkpoint. Each server stores its current checkpoint inside ZooKeeper, which captures the highest sequence number it has successfully executed. When a server starts up, it will retrieve this checkpoint and figure where to pick up from. An important nuance here is in the handling of potential data loss in Cassandra. The recovery logic checks if the local Cassandra table is empty. If an empty db is detected (implying a fresh node) despite a non-zero checkpoint, the system safely invalidates the checkpoint and replays the entire available history from the log. To support this while also abiding by the MAX_LOG_SIZE constraint, servers only delete log entries that all replicas have acknowledged executing, and they also maintain a significant safety buffer of 2500 entries. This buffer allows a recovering node to build its state from scratch purely from the log if its local persistence fails.

Tests Passed

All tests pass locally.

Challenge (Test 41)

Test 41 was particularly challenging to pass. It initially failed with 'Number of committed requests post-crash is lower compared to pre-crash' error, which suggested that the recovered data was only a fraction of the original data.

The root cause behind this was that the Cassandra db state was being wiped upon process restart. So when the servers restarted and they tried to read their checkpoint, the local database was empty, which rendered the rest of the recovery logic useless. To fix this, I modified the log pruning logic to maintain a larger safety buffer, which ensured that even if the local database is lost, the ZK log retains enough history to replay the entire test workload.